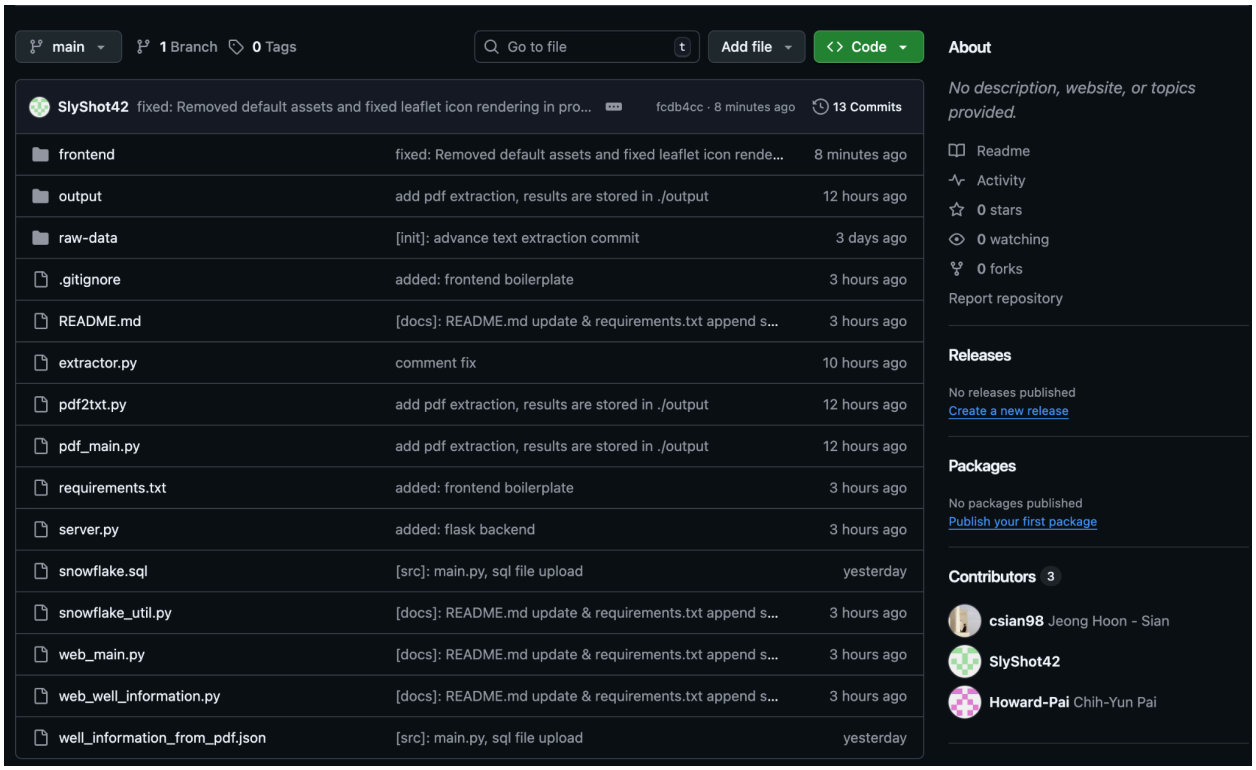
Laboratory Assignment 6

Group 15: Jeong Hoon Choi, Viraj Bansal, Chih-Yun Pai



https://github.com/csian98/advance-text-extraction

Our group decided to use the SnowFlake framework as a workspace in this laboratory.

PDF Extraction

The original PDFs contain a mix of scanned pages and text-based (digitally generated) documents. To ensure successful data extraction, we will first convert the PDFs into TXT format.

Converting PDF to TXT

Rather than applying OCR to every page, we will first evaluate the amount of text detected by PDFPlumber. If the extracted text exceeds a defined threshold, we will use PDFPlumber directly, as this approach is more accurate and computationally efficient.

OCR will only be applied to pages where the detected text falls below the threshold, indicating that the page is likely scanned or too unclear for reliable text extraction through standard parsing methods.

```
with pdfplumber.open(pdf_path) as pdf:
    total = len(pdf.pages)

    for page_num, plumber_page in enumerate(pdf.pages, start=1):
        text, method = extract_page(
            pdf_path, page_num, plumber_page, dpi, min_chars
        )
        parts.append(f"{PAGE_SEP.format(page=page_num)}\n\n{text}")
        print(f"  Page {page_num}/{total} [{method}]", end="\r")
```

Extracting Well Information

We will focus on extracting **well_name_and_number** and **API number**, as these two fields are the most critical for retrieving related information from the web in the next stage.

For extracting the well name and number, we assume that a valid well name satisfies the following conditions:

- It begins with a capitalized English word.
- It contains a number (which may include hyphenated formats such as *12-1T*).

We first attempt to locate this information immediately following a header such as **"Well Name"** or a similar label. Ideally, the value appears directly beneath or beside the header.

If the header-based extraction fails, we perform a page-wide search to identify text that matches the defined pattern. Additionally, several exclusion rules are applied to prevent capturing trailing fragments, unrelated values, or incorrectly formatted text (see code for details).

Well Name and Number					
Jefferies 5301 43-12B					
Footages	Qtr-Qtr	Section	Township	Range	
250 F S L 2510 F E L	SWSE	12	153 N	101 W	
Field	Pool	County			
Baker	Bakken	McKenzie			

24-HOUR PRODUCTION RATE

Before		After	
Oil	Bbls	Oil	Bbls
Water	Bbls	Water	Bbls
Gas	MCF	Gas	MCF

WELL NAME: Atlanta 14-6H
Atlanta 14 Well Eco Pad

Regarding the API number, we apply four strategies in priority order and stop once a valid match is found:

1. **Labeled API** – Search for explicit labels such as *"API No."* or *"API #"* followed by digits. Two safeguards are applied:
   - Skip matches that appear near a *"Well File"* listing, which may reference multiple wells.
   - Skip matches surrounded by multiple hyphenated number patterns, which may indicate a multi-well listing context.

Well File #22099 Yukon 5301 41-12T SWSW 12-153-101 API 33-053-03911
Well File #25571 Colville 5301 44-12T SESE 12-153-101 API 33-053-04981

2. Formatted pattern XX-XXX-XXXXX — Search directly for the canonical hyphenated API format. This is the most reliable method when the pattern appears clearly in the text.

API#: 33-105-02732

3. Bare 10-digit run — After applying OCR corrections (e.g., O → 0, I → 1), search for any sequence of 10 consecutive digits. To reduce false positives (such as phone numbers or ZIP codes), we validate that the first two digits correspond to a plausible U.S. state FIPS code (01–56).

```
Job Number County/State API Number
N 48 1' 29.71200" W 103 36° 16.48800" 36 153 101
```

4. Spaced/dashed fallback — Capture cases where OCR introduces extra spacing around hyphens, such as *"33 - 053 - 04856."*

```
Permit and File Number API Number Date Approved
29244 33 - 053 - 06225 8 / 26 / 2014
```

For other attributes, we primarily rely on detecting labeled headers and extracting the values that follow them.

Extracting Stimulated Information

Well-specific stimulation data is typically stored in tabular format. Once the first row of the table is detected, we extract the corresponding values from the subsequent rows.

Well Specific Stimulations

Date Stimulated	Stimulated Formation	Top (Ft)	Bottom (Ft)	Stimulation Stages	Volume	Volume Units
11/17/2011	Bakken	10882	20300	36	74826	Barrels

Type Treatment	Acid %	Lbs Proppant	Maximum Treatment Pressure (PSI)	Maximum Treatment Rate (BBLS/Min)
Sand Frac		4539124	8672	41.2

Details
40/70 white- 1818580
20/40 ceramic- 2720544

However, these tables are usually scanned pages, which introduces two key challenges:

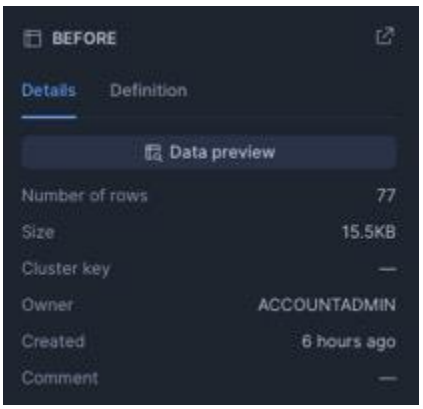
1. **OCR Errors (Typos)**
   Scanned documents frequently contain recognition errors. These are difficult to eliminate entirely unless the PDF is reprocessed at a higher resolution.

2. **Missing or Misaligned Values**
   To handle missing or ambiguous values, we apply heuristic rules to infer the correct attribute. For example:
   - Very large numeric values (e.g., >100,000) are more likely to represent Lbs rather than PSI.
   - Volume units are expected to be strings rather than integers.

```
Well Specific Stimulations
Stimulated Formation Top (Ft) | Bottom (Ft)/Stimulation Stages
41/17/2011 Bakken 4 @

Type Treatment

Details

40/70 white- 1818580
20/40 ceramic- 2720544
```

Upload to SnowFlake Database

All data was uploaded to the SnowFlake LAB_6_DB.PUBLIC.BEFORE table using the JSON file extracted from the PDF.

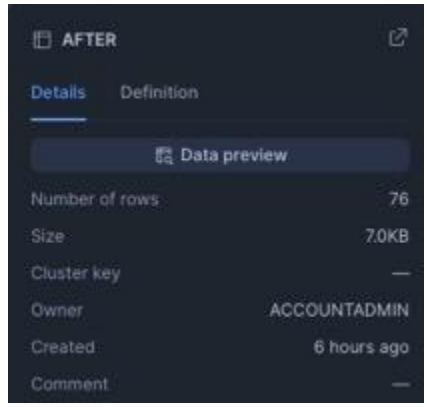


The data schema extracted as PDF is as above and consists of 19 fields including WELL_NAME and API_NO. A total of 77 WELL_NAME, API_NO pairs of data were extracted.

Additional Web Information from Web

Additional Well Information was extracted using static web scraping using Requests and BeautifulSoup in Python. WELL_NAME and API_NO were received through a query from the SnowFlake BEFORE table, and a search was performed on them.
If a single result for the WELL_NAME, API_NO pair was obtained, information was extracted from the corresponding hyperlink. If no search results were found, information was extracted with API_NO as the priority. If this also fails WELL_NAME was used.

```sql
CREATE OR REPLACE TABLE LAB_6_DB.PUBLIC.AFTER(
        WELL_NAME VARCHAR(256) NOT NULL,
        API_NO VARCHAR(256) NOT NULL,
        OPERATOR VARCHAR(256),
        WELL_STATUS VARCHAR(256),
        WELL_TYPE VARCHAR(256),
        CLOSEST_CITY VARCHAR(256),
        LATITUDE VARCHAR(256),
        LONGITUDE VARCHAR(256),
        OIL_PRODUCED VARCHAR(256),
        GAS_PRODUCED VARCHAR(256),
        PRIMARY KEY (WELL_NAME, API_NO)
);
```



Additional Information obtained via the web is stored in the SnowFlake LAB_6_DB.PUBLIC.AFTER table.
Except for OIL_PRODUCED and GAS_PRODUCED, information is always provided; the other two labels are optional for each well. Among the 77 pairs extracted from PDF, one pair failed to be extracted from the WELL_NAME, API_NO pair, and data was successfully extracted for the remaining 76.

```
~/synology-reliquia/academy/USC/MSDS/DSCI-560/laboratory/lab6/advance-text-extraction (main*) » python main.py
Upload BEFORE table [21/77]...
```

```
~/synology-reliquia/academy/USC/MSDS/DSCI-560/laboratory/lab6/advance-text-extraction (main*) » python main.py
Complete Upload BEFORE table..
Retrieve Extra Info [48/77]...
```

```
~/synology-reliquia/academy/USC/MSDS/DSCI-560/laboratory/lab6/advance-text-extraction (main*) » python main.py
Complete Upload BEFORE table..
Complete retrieve Extra Info..
Upload AFTER table [39/77]...
```

```
>>> after_df.head()
                WELL_NAME        API_NO                       OPERATOR  ...   LONGITUDE OIL_PRODUCED GAS_PRODUCED
0  Basic Game And Fish 34-3  33-053-02102              RIM OPERATING, INC.  ...  -103.645192          518          518
1  Corps Of Engineers 31-10  33-053-02148              RIM OPERATING, INC.  ...  -103.656644           64         None
2      Lewis And Clark 2-4H  33-053-02556              RIM OPERATING, INC.  ...  -103.670242          537          537
3  Wade Federal 5300 21-30H  33-053-03413  OASIS PETROLEUM NORTH AMERICA LLC  ...  -103.603100           63           12
4      Chalmers 5300 31-19H  33-053-03472  OASIS PETROLEUM NORTH AMERICA LLC  ...  -103.602355         None         None
```

Webpage and Mapping

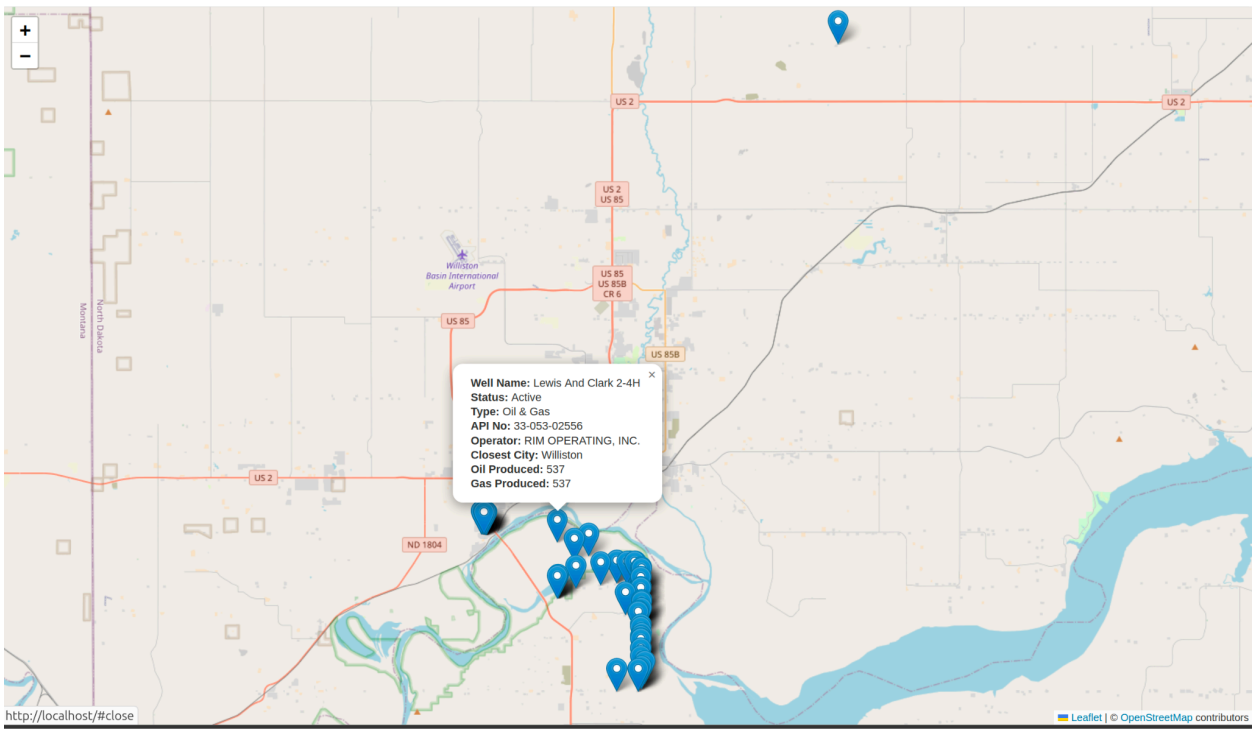
The frontend of this lab is implemented in React using the React Leaflet for rendering the map and its associated well markers as well as the popup information for each of those wells. The reason for this was due to prior experience of working in React which made it simple for us to implement. The React framework we used is Vite as this framework is fairly lightweight and it is also a framework we have extensive experience with.

Instead of directly retrieving the wells data in our frontend using a Node.js connector package, we wrote a small backend api server using Python's Flask library to prefetch the data for us. The reason for doing this was twofold:

1. We were able to avoid code duplication by utilizing the get_after() function that we already implemented in the snowflake_utils.py file
2. We were able to shift the Snowflake authentication and data fetching delays to the backend which accelerated the webpage rendering

```
(venv) viraj@virajpc:~/Documents/advance-text-extraction$ python3 server.py
Prefetching wells data...
==================================================
Done prefetching wells data.
==================================================
                 WELL_NAME        API_NO                          OPERATOR            WELL_STATUS  WELL_TYPE CLOSEST_CITY   LATITUDE    LONGITUDE OIL_PRODUCED GAS_PRODUCED
0  Basic Game And Fish 34-3  33-053-02102                RIM OPERATING, INC.                Active  Oil & Gas    Williston  48.097836  -103.645192          518          518
1  Corps Of Engineers 31-10  33-053-02148                RIM OPERATING, INC.  Plugged and Abandoned  Oil & Gas    Williston  48.094925  -103.656644           64         None
2      Lewis And Clark 2-4H  33-053-02556                RIM OPERATING, INC.                Active  Oil & Gas    Williston  48.104997  -103.670242          537          537
3  Wade Federal 5300 21-30H  33-053-03413  OASIS PETROLEUM NORTH AMERICA LLC                Active  Oil & Gas    Williston  48.047147  -103.603100           63           12
4      Chalmers 5300 31-19H  33-053-03472  OASIS PETROLEUM NORTH AMERICA LLC             Abandoned  Oil & Gas    Williston  48.057438  -103.602355         None         None
 * Serving Flask app 'server'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.0.2.15:5000
```

The rest of the frontend implementation was relatively straightforward as we were able to build on top of the setup code provided on React Leaflet's website. We set up our initial map coordinates and zoom such that all the wells in our database were clearly visible on load.



For state management, we utilized the "userImmerReducer" React hook which is part of the use-immer package. We chose to use this instead of the classic "useState" hook as "userImmerReducer" simplifies and centralizes the state management logic associated with any particular React component state variable.

As per lab instructions, we set up a local Apache Web server to serve a productionalized version of the webpage. We did run into problems with rendering the Pushpin icon for each well's

location. This turned out to be an issue with Leaflet's Icon CSS getting left out of the entire frontend build during the compilation stage. So we needed to create a small helper file called "leafletIconFix.tsx" to force include the Leaflet's Icon CSS into our frontend build after which we were successfully able to render Pushpins in the production environment.